1. In class, we said that the exact running time of an algorithm is the number of primitive operations it takes on an input of size n. (Usually, we disregard this precise running time and instead give some simpler function such that the precise function is big O if the other function.) The course textbook (Algorithms by Jeff Erickson) suggests defining the precise running time in a different way. What is that different way?

Solution: The textbook suggests identifying a specific milestone in the algorithm so that the number of times that milestone is reached will be proportional to the overall runtime, regardless of how the algorithm is actually implemented. ■

2. Consider the following basic problem. You're given an array A consisting of n integers $A[1], A[2], \ldots, A[n]$. You'd like to output a two-dimensional n-by-n array B in which $B[i, j]$ (for $i < j$) contains the sum of array entries $A[i]$ through $A[j]$–that is, the sum $A[i] + A[i+1] + \cdots + A[j]$. (The value of array entry $B[i, j]$ is left unspecified whenever $i \geq j$, so it doesn't matter what is output by those values.)

3. Give the resulting B for $A = [4, -9, 12, 2]$.

Solution:

$$B[1,2] = -5, B[1,3] = 7, B[1,4] = 9$$
$$B[2,3] = 3, B[2,4] = 5$$
$$B[3,4] = 14$$

■

```
For i = 1, 2, ..., n
   For j = i + 1, i + 2, ..., n
        Add up array entries A[i] through A[j]
        Store the result in B[i, j]
   Endfor
Endfor
```

(a) Give a function f that counts the number of primitive operations performed by the algorithm on an input of size n. This should be an exact count, not an asymptotic one!

Solution: Let's try to just count the number of additions that happen. We won't account for other assignment and retrieval operations.

As a warm up, let's ask ourselves exactly how many times the inner loop runs. The outer loop runs exactly n times, for $i = 1, 2, \ldots, n$. The inner loop runs $n - i$ times for each value of i, since j takes values from $i + 1$ through n. So the total number of times the inner loop runs is

$$(n-1) + (n-2) + (n-3) + \cdots + 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$$

by application of Euler's formula, which says that $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

Notice that we could have also expressed the number of times the inner loop runs as a double sum:

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n} 1,$$

meaning that $\sum_{i=1}^{n}\sum_{j=i+1}^{n} 1 = \frac{(n-1)n}{2}$, which we will use later.

Now that we're warmed up, let's count the number of additions.

Inside the inner loop, we need to add up the entries from $A[i]$ through $A[j]$. This requires $(j-i+1)$ additions. So the total number of additions performed by the algorithm is

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n}(j-i+1).$$

It would be fine to just look up a way to simplify this summation, but I provide a way to do so here.

We can simplify this expression as follows:

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n}(j-i+1) = \sum_{i=1}^{n}\sum_{j=i+1}^{n} j - \sum_{i=1}^{n}\sum_{j=i+1}^{n} i + \sum_{i=1}^{n}\sum_{j=i+1}^{n} 1$$

Let's tackle the easiest term first: the last term is just the number of times the inner loop runs, which we computed above to be $\frac{(n-1)n}{2}$.

The middle term is a bit trickier. Notice that for each fixed i, the inner sum $\sum_{j=i+1}^{n} i$ is just i added to itself $(n-i)$ times, or $i(n-i)$. So we have

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n} i = \sum_{i=1}^{n} i(n-i) = \sum_{i=1}^{n} ni - i^2 = n\sum_{i=1}^{n} i - \sum_{i=1}^{n} i^2.$$

We can apply Euler's formula to the first term and the sum of squares formula ($\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$) to the second term to get

$$n\sum_{i=1}^{n} i - \sum_{i=1}^{n} i^2 = n\cdot\frac{n(n+1)}{2} - \frac{n(n+1)(2n+1)}{6}.$$

The first term is similar but a bit more complicated. For each fixed i, the inner sum $\sum_{j=i+1}^{n} j$ is just the sum of the integers from $(i+1)$ through n. Notice that each of these individuals sums can be computed as the sum of i to n with the sum from i to j subtracted off. This is equal to $\sum_{j=1}^{n} j - \sum_{j=1}^{i} j$, which we can evaluate by applying Euler's formula to each term. So we have

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n} j = \sum_{i=1}^{n}\left(\sum_{j=1}^{n} j - \sum_{j=1}^{i} j\right)$$

$$
\begin{aligned}
&= \sum_{i=1}^{n} \left(\frac{n(n+1)}{2} - \frac{i(i+1)}{2} \right) \\
&= \sum_{i=1}^{n} \left(\frac{n(n+1)}{2} - \frac{i^2}{2} - \frac{i}{2} \right) \\
&= n \cdot \frac{n(n+1)}{2} - \frac{1}{2} \left(\sum_{i=1}^{n} i^2 + \sum_{i=1}^{n} i \right) \\
&= n \cdot \frac{n(n+1)}{2} - \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right)
\end{aligned}
$$

Now that we've written each term as a function of n, we could put them all together and simplify to get our final answer (but I'm not that patient). Notice that the final answer would be $\Theta(n^3)$.

■

(b) Choose a function g that has the following properties:

- The $f(n)$ you gave above is $O(g(n))$
- The $f(n)$ you gave above is $\Omega(g(n))$

Solution: As noted above, our $f(n)$ is clearly both $O(n^3)$ and $\Omega(n^3)$ since it is a polynomial of degree three. So $g(n) = n^3$. ■

(c) Although the algorithm you analyzed in parts (a)–(c) is the most natural way to solve the problem—after all, it just iterates through the relevant entries of the array B, filling a value for each—it contains some highly unnecessary sources of inefficiency. Give a different algorithm to solve this problem, with an asymptotically better running time.

Solution: The trick here is to recognize that we can reuse computation from previous steps. Rather than recompute entries $A[i]$ through $A[j-1]$, simply use what is stored in $B[i, j-1]$ and add $A[j]$ to it.

Because we want the sum from $A[i]$ through $A[j]$ inclusive, we have to initialize the first diagonal to $A[i] + A[i+1]$ separately.

```
For i = 1, 2, ..., n
    B[i, i+1] = A[i] + A[i+1]
    For j = i+2, i+3, ..., n
        B[i, j] = B[i, j-1] + A[j]
```

■

(d) What does "asymptotically better" mean?

Solution: I didn't grade this problem, since I wasn't very clear about what I wanted. I hoped you might think about how your first algorithm was $\Theta(n^3)$ but your second was $\Theta(n^2)$, and notice that those functions were different, so you could say that an algorithm with runtime $f(n) = \Theta(g(n))$ is asymptotically

better than an algorithm with runtime $f'(n) = \Theta(h(n))$ if $g(n) = \Omega(h(n))$ but $h(n) \neq \Omega(g(n))$, or something of this sort.

■